Détection et classification d'avions sur images satellites

Rapport de projet · IPSA Toulouse · module « Introduction à l'IA » (Julien Breton)

Binôme : Mathis Derory et Salomé Pinte Voisin

- **Dépôt Git :** github.com/suuuucdcz/detection-avions-mar20
- **Corpus de test annoté par le modèle :** .../detection-avions-mar20/tree/main/reports/corpus_test_annote

Le corpus annoté contient les 131 images de test annotées par notre modèle : un fichier XML par image (`annotations/`, au format Pascal VOC du dataset fourni), un aperçu avec les boîtes dessinées (`apercus/`), le tableau de toutes les détections (`detections.csv`) et une notice (`LISEZMOI.md`).

1. Le problème et les données

L'application doit **localiser les avions** dans une image satellite et **déterminer leur type**, parmi 20 codés A1 à A20. Le dataset fourni compte **1 331 images** (le plus souvent 800 × 800 pixels) et **7 870 avions** annotés. Les types sont très déséquilibrés : de 107 avions (A6) à 785 (A16). Aucune donnée extérieure n'a été ajoutée.

On a mis de côté nos propres jeux de validation et de test. Le découpage est fait **par image**, il est **stratifié** par type, et les quasi-doublons restent dans le même jeu, pour éviter toute fuite de données :

Jeu	Images	Avions	Rôle
Entraînement	1 070	6 295	apprendre
Validation	130	787	choisir les réglages et comparer les modèles
Test	131	788	mesurer le score final ; c'est le corpus annoté

Aucun choix n'a été fait sur le jeu de test : tous les réglages ont été figés sur la validation avant de le regarder.

2. L'architecture : deux étages spécialisés

```
image satellite
   │
   ▼
1. DÉTECTEUR  YOLO11n, une seule classe « avion »  ──►  une boîte et une confiance par avion
   │
   ▼
   vignette : un carré de 2 × le plus grand côté de la boîte, centré sur l'avion
   (la marge sert aux rotations ; le classifieur en regarde le centre, en 224 × 224)
   │
   ▼
2. CLASSIFIEUR  ResNet-18  ──►  un type parmi A1…A20, et une confiance
   │
   ▼
confiance finale = confiance du détecteur × confiance du classifieur
le modèle répond si elle atteint 0,5 (seuil choisi sur la validation)
```

- **Le détecteur** n'a qu'une question à résoudre, « avion ou pas ? ». Tous les avions de l'entraînement lui servent d'exemples, quel que soit leur type, si bien que le déséquilibre entre les types ne le gêne pas.
- **Le classifieur** voit chaque avion en gros plan, et on peut le faire tourner dans tous les sens pendant l'entraînement : vu du ciel, un avion n'a pas d'orientation privilégiée.
- **Le produit des deux confiances** sert de second filtre : une boîte posée sur le fond ne ressemble à aucun type, sa confiance finale reste basse. En validation, à seuil égal (0,25), ce filtre fait passer les fausses alarmes de 43 à 16, sans perdre un seul avion bien classé ; au seuil retenu de 0,5, il n'en reste que 4.
- **Un témoin** vérifie ce choix : un YOLO11n à 20 classes, qui fait tout en une seule passe.

Les deux réseaux partent de **poids pré-entraînés** (transfer learning, autorisé par le sujet) : COCO pour YOLO11n, ImageNet pour le ResNet-18.

3. L'entraînement

Les entraînements ont été faits sur **Google Colab** (GPU T4), avec les mêmes scripts et les mêmes fichiers de réglages (`configs/`) que sur le PC.

- **Le détecteur** : images en 800 pixels pour ne pas rétrécir les plus petits avions (38 pixels), retournements, et early stopping (meilleure epoch : 52, arrêt à la 67e). En validation, il retrouve 99,7 % des avions (rappel de 0,997).
- **Le classifieur** :
  - rotations de n'importe quel angle, retournements, luminosité et contraste ±20 %, cadrage légèrement décalé, pour imiter les boîtes du détecteur ;
  - une perte pondérée pour aider les types rares ;
  - les 3 premières epochs avec le backbone gelé ;
  - l'optimiseur AdamW, avec un learning rate de départ de 0,0003 ;
  - un early stopping sur la macro-F1 de validation.
- **Les modèles de comparaison** : une forêt aléatoire sur des caractéristiques faites à la main (HOG et taille), un petit CNN créé de zéro, et le même ResNet-18 sans pré-entraînement.

4. Les résultats, sur le jeu de test

Le système complet (131 images, 788 avions ; décomptes au seuil de confiance de 0,5, mAP50 calculée sur tous les seuils) :

Système	Justes	Mauvais type	Fausses alarmes	Oubliés	Précision	Rappel	F1	mAP50
Deux étages (le nôtre)	**729**	**9**	**3**	50	**0,984**	0,925	**0,954**	**0,957**
Témoin YOLO à 20 classes	717	49	19	22	0,913	0,910	0,912	0,940

Notre système fait **5 fois moins d'erreurs de type** et **6 fois moins de fausses alarmes** que le témoin. Le témoin est environ 3 fois plus rapide (72 ms contre 202 ms par image, sur le processeur), et il oublie moins d'avions.

Le classifieur seul (macro-F1 sur les vignettes de test) :

Forêt aléatoire	Petit CNN	ResNet-18 sans pré-entraînement	ResNet-18 pré-entraîné
0,625	0,567	0,949	**0,958**

Le Deep Learning surpasse nettement le Machine Learning classique. Le transfer learning réduit les erreurs d'environ 30 % (30 au lieu de 44). La forêt aléatoire illustre l'overfitting : 100 % de bonnes réponses sur l'entraînement, 69 % en validation. Le petit CNN illustre l'underfitting.

L'analyse des erreurs :

- **Les 3 fausses alarmes ne sont pas des boîtes sur le fond.** Deux sont des doublons, sur l'image 3734 : deux avions y ont chacun reçu une 2e boîte, que la suppression des doublons (NMS, au-delà de 70 % de recouvrement) n'a pas éliminée (recouvrements de 67 et 69 %). La troisième est un vrai avion, coupé par le bord de l'image 3607, que le dataset n'a pas annoté.
- **Sur les 50 avions oubliés, le détecteur en avait trouvé 49.** Pour 47 d'entre eux, il en était sûr, mais le classifieur hésitait sur le type (confiance médiane de 0,46). Le levier d'amélioration est donc le classifieur, pas le détecteur.
- **Les erreurs de type** touchent surtout des avions qui se ressemblent : A13 pris pour A15 (5 des 9 erreurs), A11 pris pour A14 (2), et une fois A1 pris pour A19 et A19 pour A20.

5. L'application et le corpus annoté

- **L'interface** (`uv run python scripts/app.py`, bibliothèque Gradio) :
  - elle propose les 131 images de test, sans dépôt d'image extérieure ;
  - « Détecter les avions » affiche les réponses du modèle ;
  - « Corriger avec la vérité » marque chaque réponse juste ou fausse, avec la raison, et montre les avions oubliés ;
  - un **F1 cumulé** se met à jour à chaque image corrigée ; après les 131, il vaut exactement 0,954 ;
  - tout tourne sur le processeur, en environ 0,2 seconde par image.
- **Le corpus annoté** (`scripts/annotate_corpus.py`) :
  - les fichiers XML reprennent le format du dataset fourni ;
  - chaque avion y reçoit en plus une balise `<score>` (la confiance finale) et une balise `<a_verifier>` ;
  - un avion est « à vérifier » quand le détecteur est sûr de voir un avion, mais que le type est incertain : c'est un humain qui tranche. On compte 741 avions sûrs et 61 boîtes à vérifier.

6. Limites et pistes

- **Des scores sans doute un peu optimistes.** Les images de test viennent des mêmes bases aériennes que l'entraînement : sur des bases jamais vues, le score serait sans doute plus bas.
- **Des scores imprécis pour les types rares** (A6, A9, A15), qui n'ont que 3 images de test chacun.
- **Des pistes, à régler sur la validation et non sur le test** :
  - un seuil de 0,4, qui était un peu meilleur en validation ;
  - la moyenne des réponses du classifieur sur plusieurs rotations ;
  - une suppression des doublons plus stricte ;
  - un classifieur plus gros.
- **Le déploiement** dans un conteneur Docker reste à faire.

Les choix et leurs justifications sont détaillés dans `docs/journal_decisions.md` (décisions D1 à D21).